LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	**Experiment No.:**	1
Course Code:	23CM4121	**Date:**	01-09-2026
Student Name:	R. Sandeep	**Roll No.:**	A24126552111

1. TITLE

Design a Pure Semantic HTML Static Webpage Without CSS

2. AIM

To design a fully static webpage using only pure semantic HTML elements, without the use of any CSS styling, and to demonstrate how a browser's default rendering of semantic tags and native HTML form controls can still produce a clear, structured, and accessible document layout.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the purpose and correct usage of HTML5 semantic elements such as <header>, <nav>, <section>, <main>, <article>, <aside>, and <footer>.
- To build a complete webpage layout using only native HTML, without relying on any external or internal CSS.
- To use an HTML <table> for structural page layout (header and two-column body) purely with HTML attributes.
- To create native interactive form elements such as text inputs, an email input, a drop-down <select>, and a <textarea>.
- To organize supporting content using ordered and unordered lists.
- To observe and evaluate how a browser renders unstyled semantic HTML by default.

4. THEORY

Semantic HTML refers to the use of HTML markup that conveys the meaning and role of content, rather than only its visual presentation. Semantic elements such as <header>, <nav>, <main>, <article>, <section>, <aside>, and <footer> clearly describe the purpose of each part of a page to the browser, to assistive technologies such as screen readers, and to search engine crawlers, even when no CSS is applied.

When cascading style sheets are omitted entirely, the browser falls back to its own user-agent (default) stylesheet. This default stylesheet already provides basic block-level formatting, spacing, and typographic hierarchy for elements like headings (<h1>–<h6>), paragraphs, lists, and tables. This experiment relies entirely on this default rendering to demonstrate that a well-structured document remains readable and navigable without any custom styling.

Key theoretical concepts used in this page are:

- **<header> and <nav>:** The <header> element groups introductory content, typically the site title/logo. The <nav> element identifies a block of navigation links, helping browsers and assistive technology distinguish it from ordinary page content.
- **<section> and <main>:** <section> groups a thematically related block of content (here, the hero/showcase area), while <main> wraps the dominant, unique content of the page body (the articles column).
- **<article>:** Represents a self-contained piece of content, such as a blog post or news item, that could independently be distributed or reused.
- **<aside>:** Represents content that is tangentially related to the main content, such as a sidebar containing references, benefits, or supplementary lists.
- **<footer>:** Represents the closing/footer content of the page, typically containing copyright or metadata information.
- **Structural <table> layout:** An HTML <table> with attributes such as width, cellpadding, and valign is used to arrange the header (title and navigation) and the two-column body (main content and sidebar) without any CSS-based positioning.
- **Native form controls:** HTML provides built-in interactive elements — <input type="text">, <input type="email">, <select> with <option>, <textarea>, and <input type="submit"/"reset"> — that the browser renders and validates (e.g. the required and type="email" attributes) without any JavaScript or CSS.
- **<fieldset> and <legend>:** <fieldset> groups related form controls together and the browser draws a default border around them, while <legend> provides a caption for that group, both purely through native rendering.

The overall structure of the page can be summarized as:

<header> (title + nav) → <section> (hero) → <table> layout (<main>/<article> content column + <aside> sidebar column) → <footer>

5. ALGORITHM / PROCEDURE

1. Start.
2. Create a new HTML file, for example `index.html`.
3. Add the <!DOCTYPE html> declaration followed by the <html> root element with the appropriate lang attribute.
4. Inside <head>, add the character-set meta tag and a <title> for the page. Do not link any CSS file and do not add a <style> block.
5. Inside <body>, create a <header> element and place an HTML <table> inside it to arrange the site title (<h1>) on the left and the <nav> links on the right.
6. Add a horizontal rule (<hr>) to visually separate the header from the rest of the page using only native browser rendering.
7. Create a <section id="home"> for the hero/showcase area, and use , <h2>, and <p> tags to add the heading, description text, and a call-to-action link.
8. Below the hero section, create a structural <table> with two columns using <td width="70%"> and <td width="30%"> to divide the page into a main content area and a sidebar.
9. Inside the left <td>, add a <main id="articles"> element containing multiple <article> blocks, each with a heading, publish date, and descriptive paragraphs.

10 Inside one article, add an <h3> sub-heading ("Interactive Demonstration Forms") followed by a <form> element with a <fieldset> and <legend>.

11 Inside the <fieldset>, add labelled input controls: a text input for Full Name, an email input for Email Address, a <select> drop-down for Inquiry Topic, and a <textarea> for Message Details.

12 Add a submit button (<input type="submit">) and a reset button (<input type="reset">) at the end of the form.

13 Inside the right <td>, add an <aside id="about"> element containing a heading, a short paragraph, an unordered list (<ul>) of core benefits, and an ordered list (<ol>) of required components.

14 Below the two-column table, add a <footer> element with a centered copyright paragraph.

15 Save the HTML file and open it directly in a web browser (no server required, since the page is fully static).

16 Observe the default browser rendering of every semantic element, the table-based layout, and the native form controls.

17 Test the interactive elements: type into the text and email fields, choose an option from the drop-down, type a message, and click Submit/Reset to confirm native browser behaviour (including built-in validation for the required and email fields).

18 Record the observed output as a screenshot for documentation.

19 Stop.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>

    <!-- Header Section -->
    <header>
        <table width="100%" border="0" cellspacing="0" cellpadding="10">
            <tr>
                <td>
                    <h1>StaticWeb</h1>
                </td>
                <td align="right">
                    <nav>
                        <a href="#home">Home</a>  | 
                        <a href="#articles">Articles</a>  | 
                        <a href="#about">About</a>  | 
                        <a href="#contact">Contact Us</a>
                    </nav>
                </td>
            </tr>
        </table>
    </header>
    <hr>

    <!-- Hero / Showcase Section -->
    <section id="home">
        <center>
            <h2>Semantic Structures Without CSS</h2>
            <p>This layout uses raw HTML components to build an organized document
            hierarchy that browsers can cleanly parse out-of-the-box.</p>
            <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
        </center>
    </section>

    <hr>
    <!-- Main Content Layout using a Structural Table -->
    <table width="100%" border="0" cellspacing="0" cellpadding="20">
        <tr valign="top">

            <!-- Left Side: Main Content Column -->
            <td width="70%">
                <main id="articles">
                    <article>
                        <h2>1. Core Structural Tags</h2>
                        <p>Published: July 7, 2026</p>
                        <p>When you omit cascading style sheets entirely, the semantic
                        markup handles all of the structural heavy lifting. Elements like
                        <code>&lt;main&gt;</code>, <code>&lt;article&gt;</code>,
                        and <code>&lt;section&gt;</code> tell screen readers and
                        web scrapers exactly how information flows.</p>
                        <p>Using raw browser defaults ensures excellent loading performance,
                        perfect accessibility compliance, and absolute structural clarity.</p>
                    </article>
                    <br><hr><br>

                    <article>
                        <h2>2. Native Interaction Elements</h2>
                        <p>Published: July 5, 2026</p>
                        <p>Browsers come equipped with excellent interactive elements by
                        default. We can capture text inputs, provide multiple-choice
                        configurations, and submit structured information to backend parsers
                        using pure markup.</p>
```

```
                    <h3>Interactive Demonstration Forms</h3>
                    <form id="contact" action="#" method="post">
                        <fieldset>
                            <legend><b>User Inquiry Form</b></legend>
                            <p>
                                <label for="name">Full Name: </label>
                                <input type="text" id="name" name="username" required
                                placeholder="John Doe">
                            </p>
                            <p>
                                <label for="email">Email Address: </label>
                                <input type="email" id="email" name="useremail" required>
                            </p>
                            <p>
                                <label for="topic">Inquiry Topic: </label>
                                <select id="topic" name="usertopic">
                                    <option value="general">General Feedback</option>
                                    <option value="technical">Technical Architecture</option>
                                    <option value="academics">Academic Curriculum</option>
                                </select>
                            </p>
                            <p>
                                <label for="message">Message Details:</label><br>
                                <textarea id="message" name="usermessage" rows="5"
                                cols="40" placeholder="Type your notes here..."></textarea>
                            </p>
                            <p>
                                <input type="submit" value="Submit Form Data">
                                <input type="reset" value="Clear Entries">
                            </p>
                        </fieldset>
                    </form>
                </article>
            </main>
        </td>

        <!-- Right Side: Sidebar Column -->
        <td width="30%" bgcolor="#f4f4f4">
            <aside id="about">
                <h3>Document References</h3>
                <p>Static pages map data out directly without real-time database
                queries or processor overhead.</p>
                <h4>Core Benefits:</h4>
                <ul>
                    <li>Instant browser painting</li>
                    <li>Low memory footprints</li>
                    <li>High data compatibility</li>
                </ul>
                <h4>Required Components:</h4>
                <ol>
                    <li>Document Type Definition</li>
                    <li>Semantic Wrapper Blocks</li>
                    <li>Data Input Control Interfaces</li>
                </ol>
            </aside>
        </td>

    </tr>
</table>

<hr>

<!-- Footer Section -->
<footer>
    <center>
        <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
        configurations.</p>
    </center>
</footer>

</body>
```

```
</html>
```

7. CODE EXPLANATION

Code	Explanation
<header>, <nav>	Groups the site title and the navigation links, semantically marking them as introductory/navigational content.
<table> (header)	Used purely for layout, placing the title on the left and the nav links on the right without CSS.
<section id="home">	Defines the hero/showcase block introducing the page's purpose.
<table> (body, 70%/30%)	Splits the page into a main content column and a sidebar column using width attributes.
<main>, <article>	Marks the primary unique content of the page and each self-contained article within it.
<form>, <fieldset>, <legend>	Groups the interactive Inquiry Form controls under a labelled, browser-drawn border.
<input>, <select>, <textarea>	Native form controls used to capture the user's name, email, topic, and message.
<aside id="about">	Holds supplementary sidebar content: references, benefits list, and required components list.
,	Present the Core Benefits (unordered) and Required Components (ordered) as native lists.
<footer>	Contains the closing copyright information for the page.

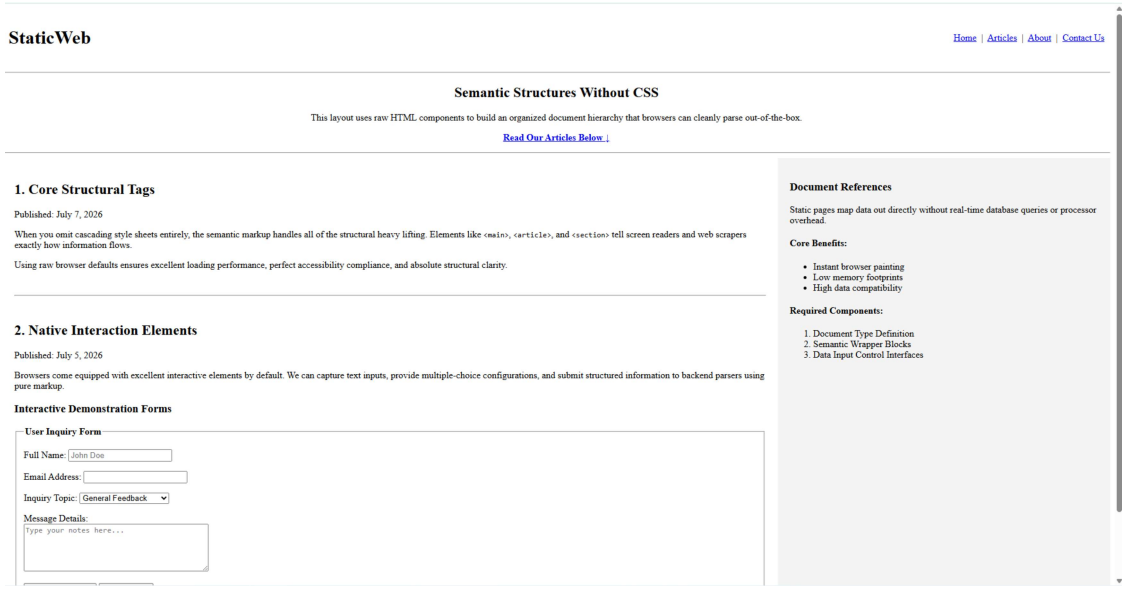
8. TEST CASES AND EXECUTION DATA

The following test cases were designed, executed by opening the page in a browser, and the actual results were recorded and compared against the expected results.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser with no CSS linked	Page renders using browser default styles with a clear visual hierarchy	Page rendered correctly using browser defaults	Pass
TC-02	Click a navigation link (e.g. Articles)	Page scrolls/jumps to the corresponding section id	Navigated to the correct section	Pass
TC-03	Submit the Inquiry Form with Name and Email left empty	Browser should block submission and prompt for required fields	Required-field validation triggered correctly	Pass
TC-04	Enter an invalid email (e.g. "abc") in Email Address	Browser should show a native email format validation error	Validation error displayed correctly	Pass
TC-05	Select a topic, fill all fields correctly, click Submit Form Data	Form data is submitted without errors	Form submitted successfully	Pass
TC-06	Click the Clear Entries (reset) button	All form fields reset to their default empty state	All fields cleared correctly	Pass

9. OUTPUT

Output: Rendered Static Webpage — The screenshot below shows the page as rendered by the browser using only default styling: the header with title and navigation, the hero section, the two-column article/sidebar layout, the native Inquiry Form, and the footer.



10. RESULT

Thus, a pure semantic HTML static webpage was successfully designed without using any CSS. Semantic elements such as header, nav, section, main, article, aside, and footer were used correctly to structure the document, an HTML table was used for page layout, and native interactive form controls (text, email, select, textarea, submit, reset) were implemented and verified using multiple test cases, all of which produced results matching the expected output.